

Machine Learning

Chapter 18.1-18.3

Some material adopted from notes
by Chuck Dyer

Today's Class

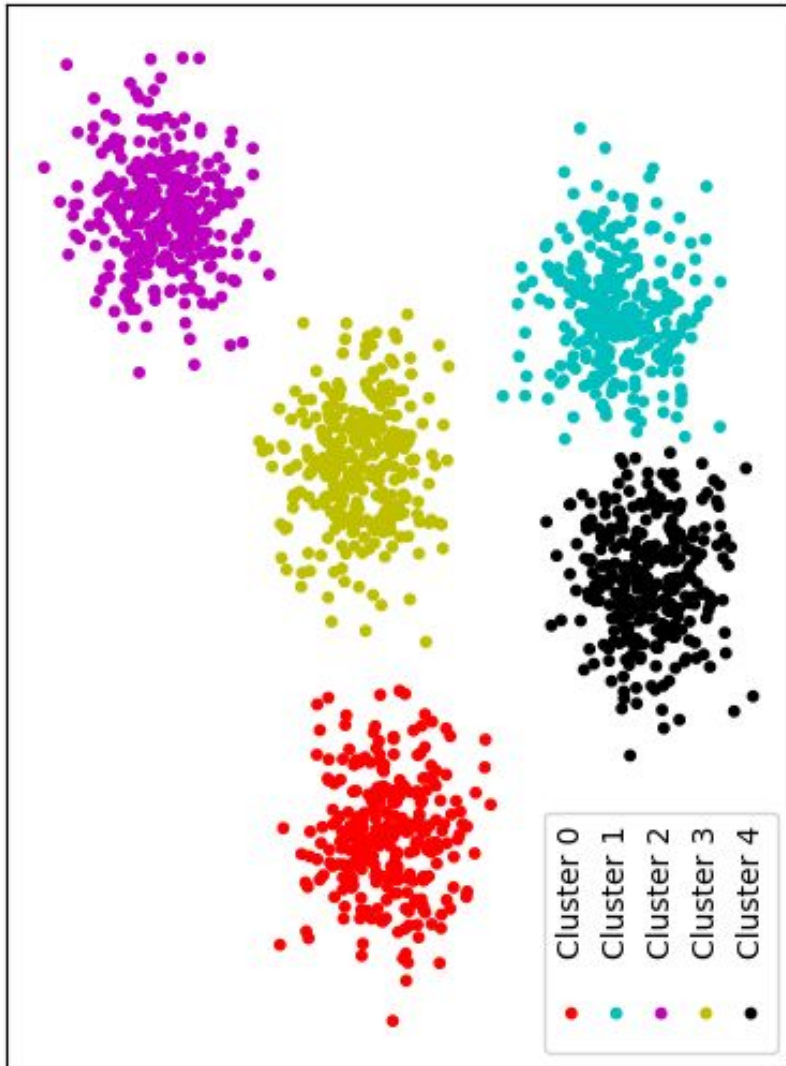
- Machine learning
 - What is ML?
 - Inductive learning
 - Supervised
 - Unsupervised
 - Decision trees
- Later we'll cover Bayesian learning, naïve Bayes, and BN learning

Why Learn?

- Understand and improve efficiency of human learning
 - Use to improve methods for teaching and tutoring people (e.g., better computer-aided instruction)
- Discover new things or structure that were previously unknown to humans
 - Examples: data mining, scientific discovery
- Fill in skeletal or incomplete specifications about a domain
 - Large, complex AI systems cannot be completely derived by hand and require dynamic updating to incorporate new information.
 - Learning new characteristics expands the domain or expertise and lessens the “brittleness” of the system
- Build software agents that can adapt to their users or to other software agents

Major Paradigms of Machine Learning

- **Rote learning** – One-to-one mapping from inputs to stored representation. “Learning by memorization.” Association-based storage and retrieval.
- **Induction** – Use specific examples to reach general conclusions
- **Clustering** – **Unsupervised** identification of natural groups in data
- **Analogy** – Determine correspondence between two different representations
- **Discovery** – Unsupervised, specific goal not given
- **Genetic algorithms** – “Evolutionary” search techniques, based on an analogy to “**survival of the fittest**”
- **Reinforcement** – Feedback (positive or negative reward) given at the end of a sequence of steps



Classify



Regression



Mean
Learner



Nearest
Neighbors



Regress...
Tree



Random
Forest ...



SVM
Regress...



Linear
Regress...



AdaBoost



Stochas...
Gradien...



Univariate
Polyno...



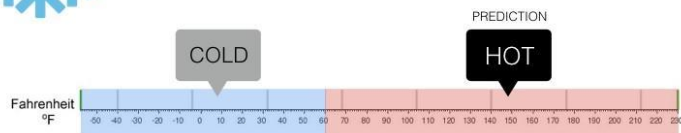
Regression

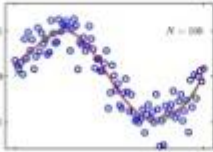
What is the temperature going to be tomorrow?



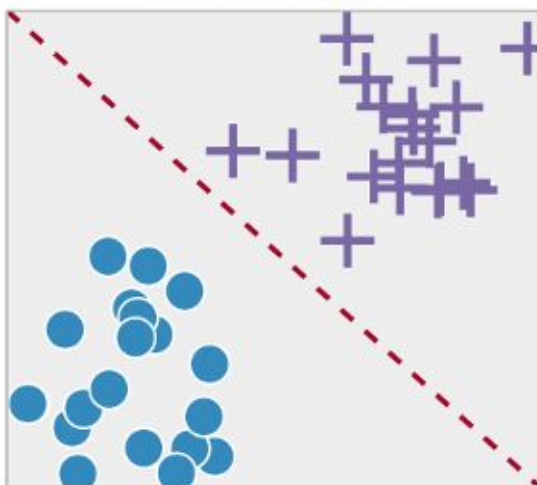
Classification

Will it be Cold or Hot tomorrow?

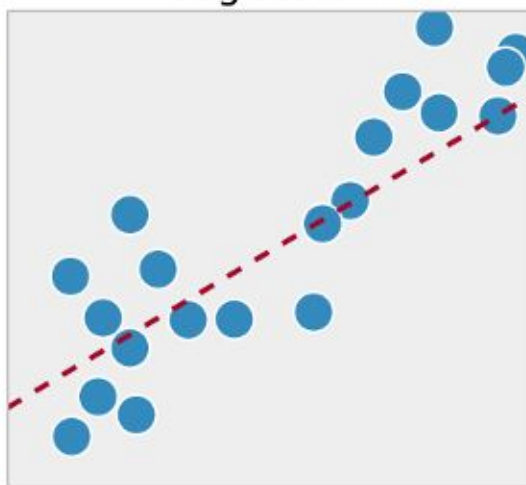


Predictive methods	Descriptive methods
Classification  Learns a method for predicting the instance class from pre-labeled (classified) instances	Clustering  Finds "natural" grouping of instances given un-labeled data
Regression  An attempt to predict a continuous attribute	Association Rules  Method for discovering interesting relations between variables in large DBs

Classification



Regression



Classification Learning: Definition

- Given a collection of records (*training set*)
 - Each record contains a set of *attributes*, one of the attributes is the *class*
- Find a *model* for the class attribute as a function of the values of the other attributes
- Goal: previously unseen records should be assigned a class as accurately as possible
 - Use *test set* to estimate the accuracy of the model
 - Often, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it

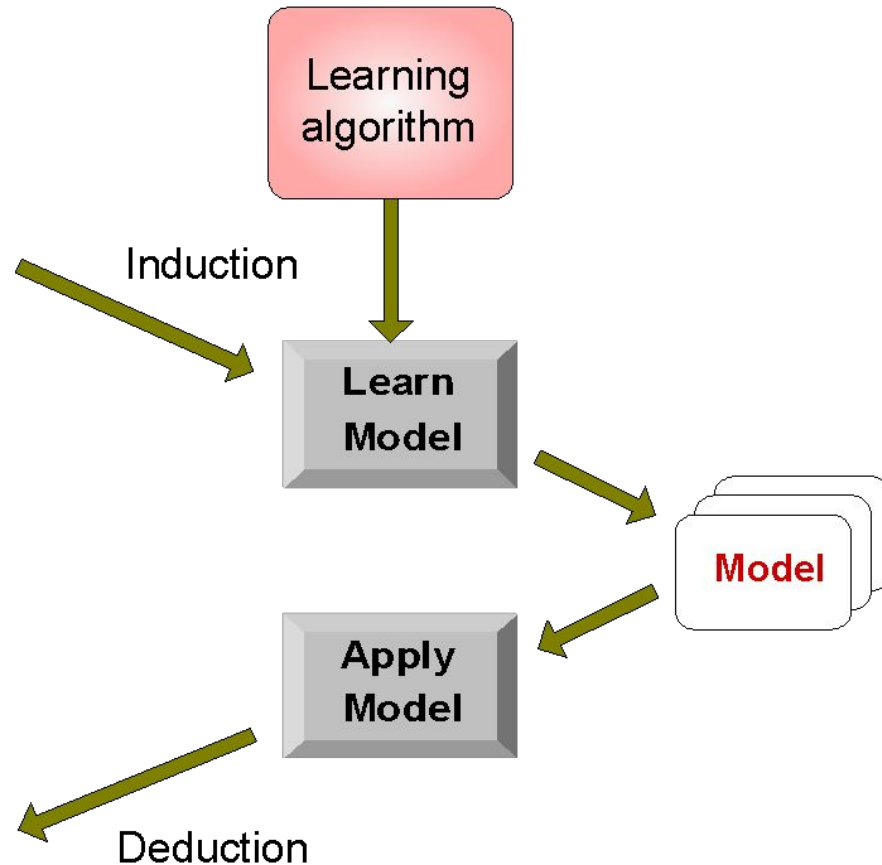
Illustrating Classification Learning

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

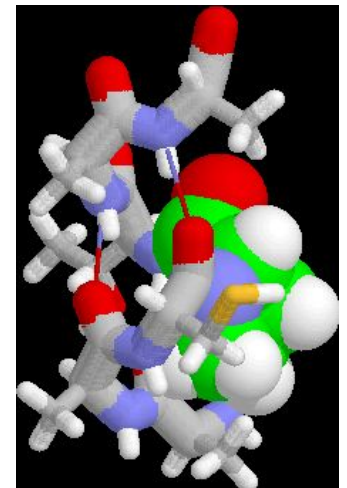
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set

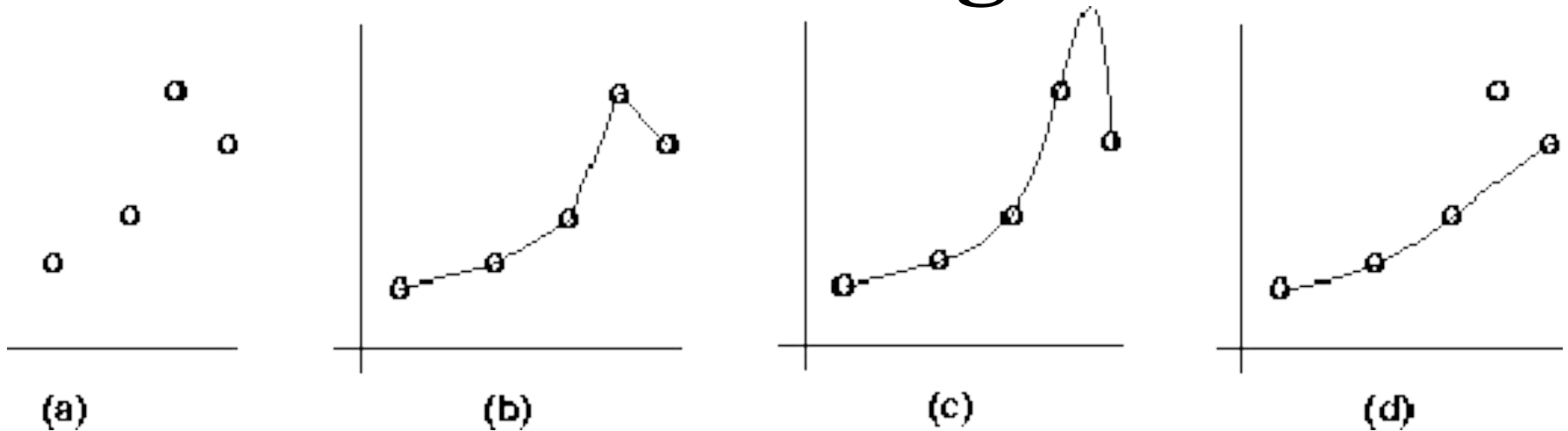


Examples of Classification Task

- Predicting tumor cells as benign or malignant
- Classifying credit card transactions as legitimate or fraudulent
- Classifying secondary structures of protein as alpha-helix, beta-sheet, or random coil
- Categorizing news stories as finance, weather, entertainment, sports, etc.



Inductive Learning and Bias



- Suppose that we want to learn a function $f(x) = y$ and we are given some sample (x,y) pairs, as in figure (a)
- There are several hypotheses we could make about this function, e.g.: (b), (c) and (d)
- A preference for one over the others reveals the **bias** of our learning technique, e.g.:
 - prefer piece-wise functions (b)
 - prefer a smooth function (c)
 - prefer a simple function and treat outliers as noise (d)

Inductive Learning as Search

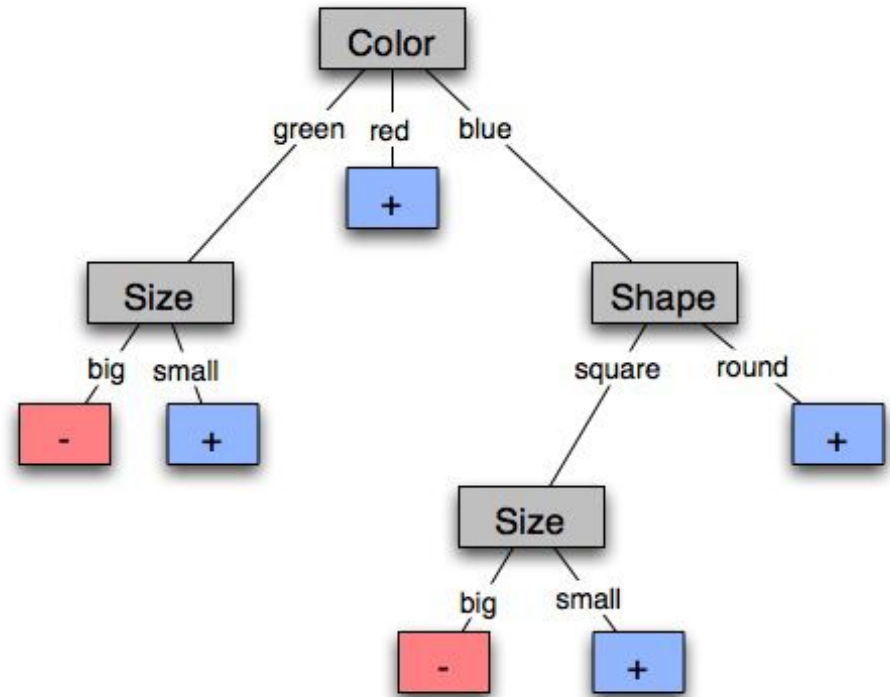
- Instance space I defines the language for the training and test instances
 - Typically, but not always, each instance $i \in I$ is a feature vector
 - Features are also sometimes called attributes or variables
 - $I: V_1 \times V_2 \times \dots \times V_k, i = (v_1, v_2, \dots, v_k)$
- Class variable C gives an instance's class (to be predicted)
- Model space M defines the possible classifiers
 - $M: I \rightarrow C, M = \{m_1, \dots, m_n\}$ (possibly infinite)
 - Model space is sometimes, but not always, defined in terms of the same features as the instance space
- Training data can be used to direct the search for a good (consistent, complete, simple) hypothesis in the model space

Model Spaces

- **Decision trees**
 - Partition the instance space into axis-parallel regions, labeled with class value
- Nearest-neighbor classifiers
 - Partition the instance space into regions defined by the centroid instances (or cluster of k instances)
- Bayesian networks (probabilistic dependencies of class on attributes)
 - Naïve Bayes: special case of BNs where class $\rightarrow$ each attribute
- Neural networks
 - Nonlinear feed-forward functions of attribute values
- Support vector machines
 - Find a separating plane in a high-dimensional feature space
- Associative rules (feature values $\rightarrow$ class)
- First-order logical rules

Learning Decision Trees

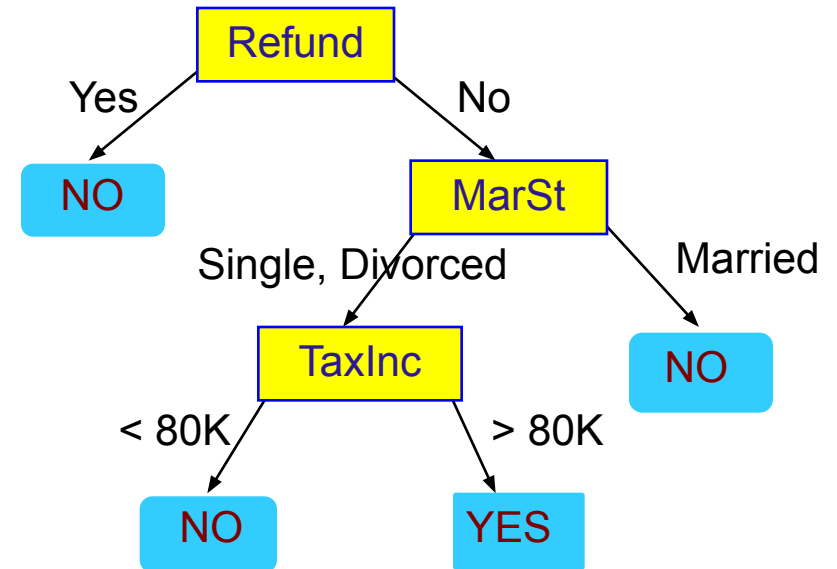
- Goal: Build a **decision tree** to **classify** examples as **positive** or **negative** instances of a concept using supervised learning from a training set
- A **decision tree** is a tree where
 - each **non-leaf node** has associated with it an attribute (**feature**)
 - each leaf node has associated with it a **classification** (+ or -)
 - each arc has associated with it one of the possible values of the attribute at the node from which the arc is directed
- Generalization: allow for >2 classes
 - e.g., {sell, hold, buy}



Example of a Decision Tree

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data



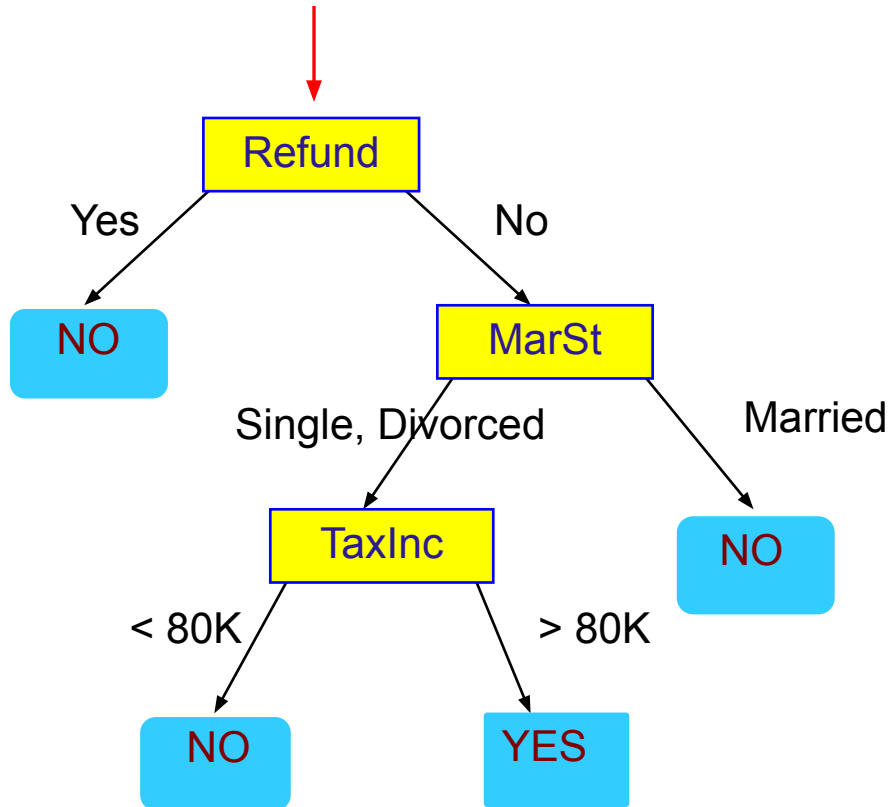
Model: Decision Tree

Apply Model to Test Data

Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

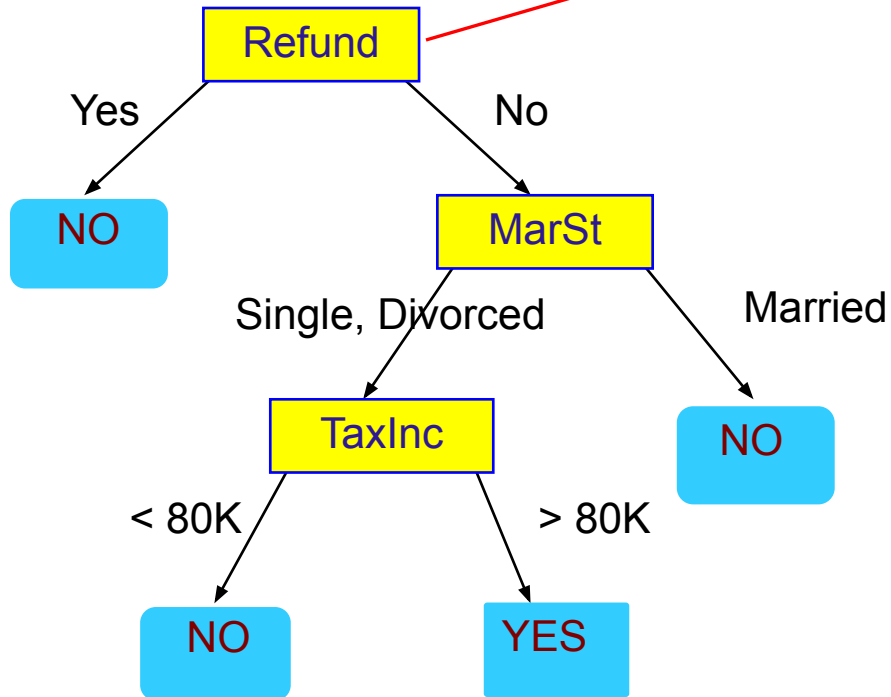
Start at the root of tree



Apply Model to Test Data

Test Data

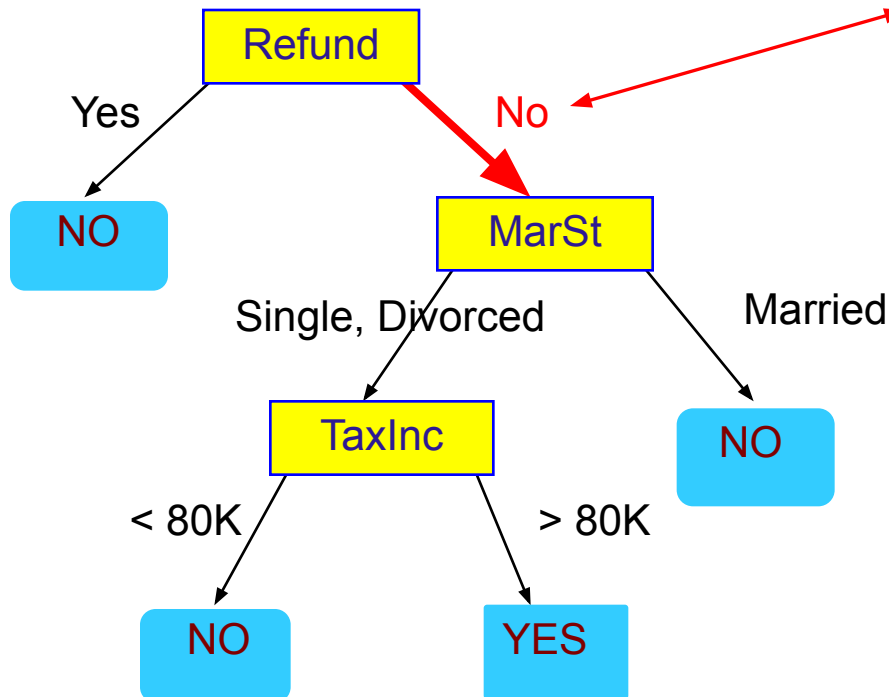
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

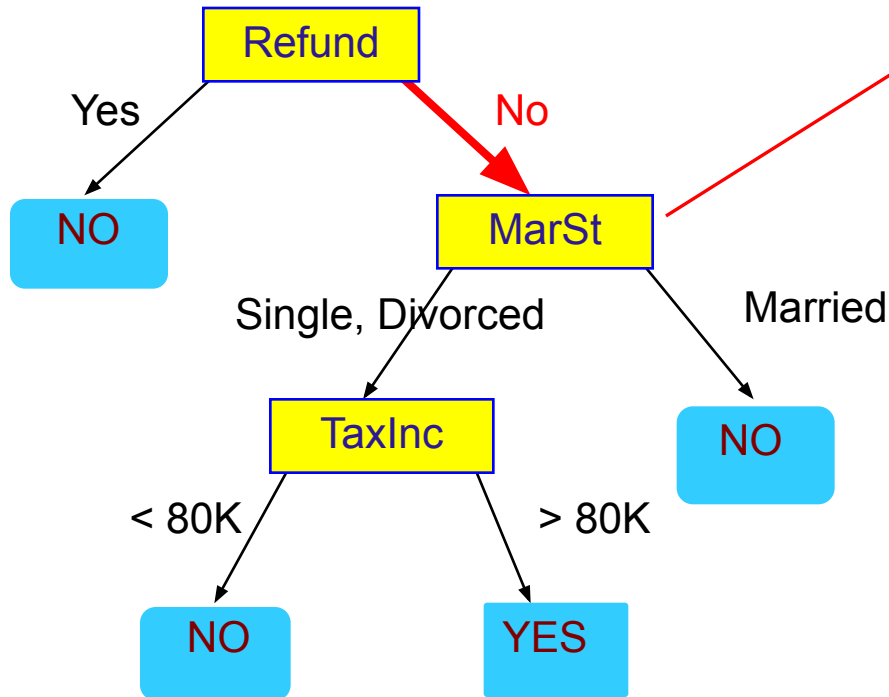
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

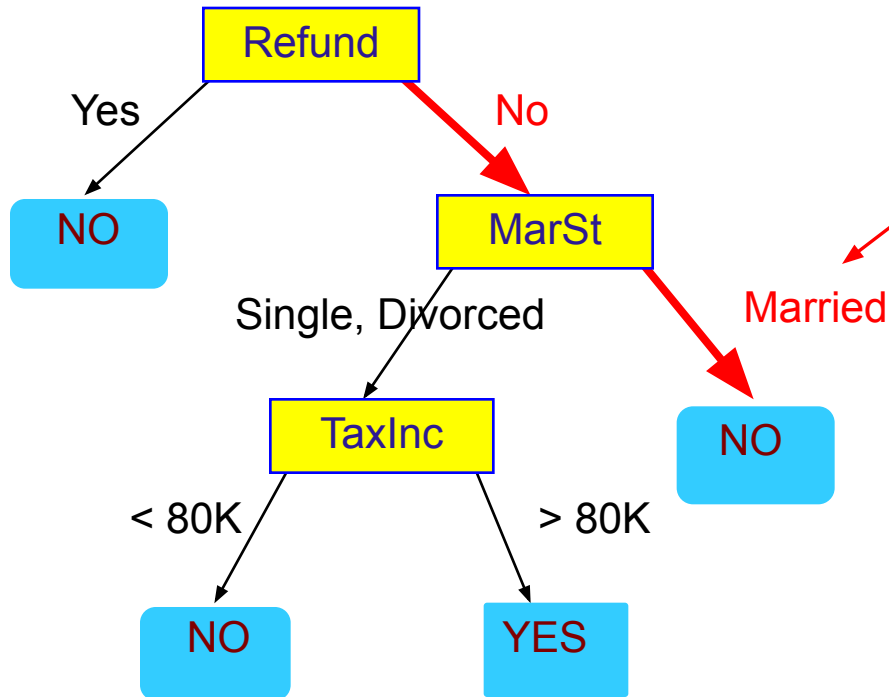
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

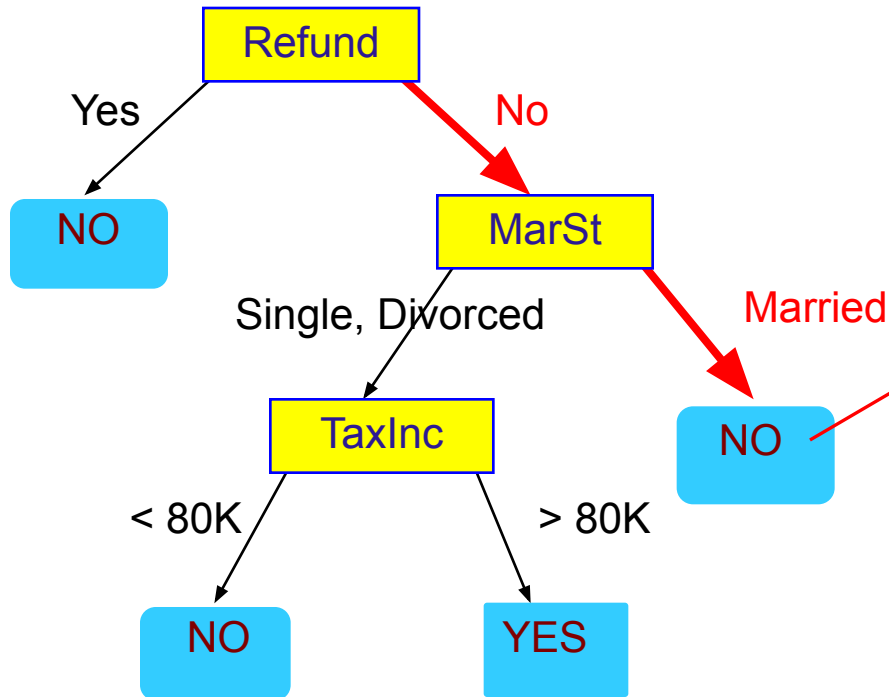
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Assign Cheat to "No"

Information Theory

- Information is measured in bits
- Information conveyed by a message depends on its probability
- With n equally probable possible *messages*, the probability p of each is $1/n$
- Information conveyed by message is $\log_2(n) = -\log_2(p)$
 - e.g., with 16 messages, then $\log_2(16) = 4$ and we need 4 bits to identify/send each message
- Given probability distribution for n messages $P = (p_1, p_2, \dots, p_n)$, the information conveyed by distribution (aka *entropy* of P) is:
$$I(P) = -(p_1 * \log_2(p_1) + p_2 * \log_2(p_2) + \dots + p_n * \log_2(p_n))$$

probability of msg 2

info in msg 2



Entropy

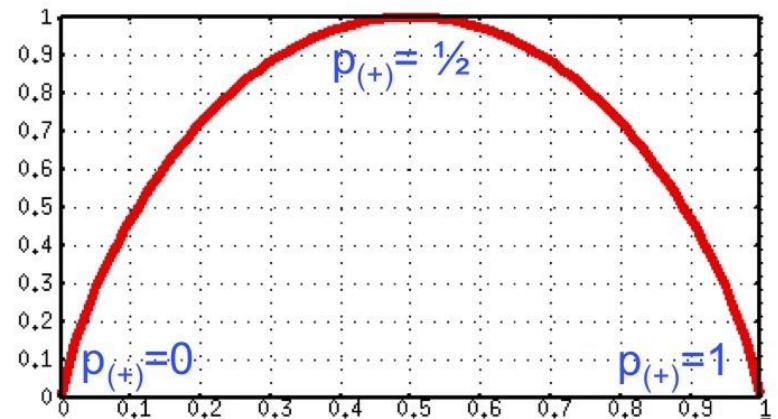
- Entropy: $H(S) = -p_{(+)} \log_2 p_{(+)} - p_{(-)} \log_2 p_{(-)}$ bits
 - S ... subset of training examples
 - $p_{(+)} / p_{(-)}$... % of positive / negative examples in S
- Interpretation: assume item X belongs to S
 - how many bits need to tell if X positive or negative

- impure (3 yes / 3 no):

$$H(S) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1 \text{ bits}$$

- pure set (4 yes / 0 no):

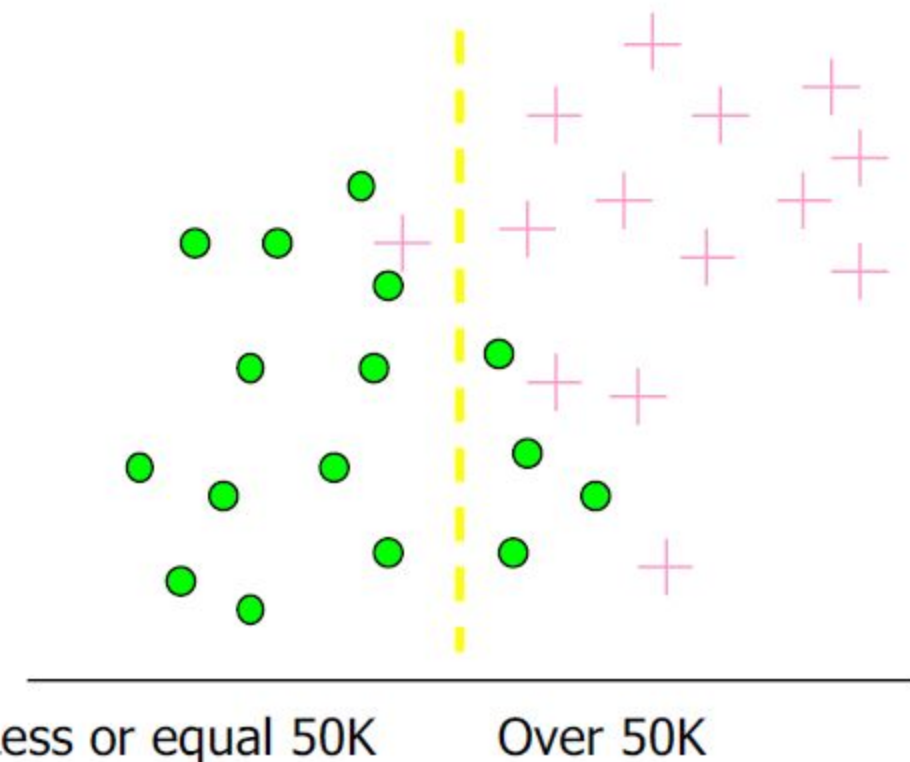
$$H(S) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0 \text{ bits}$$



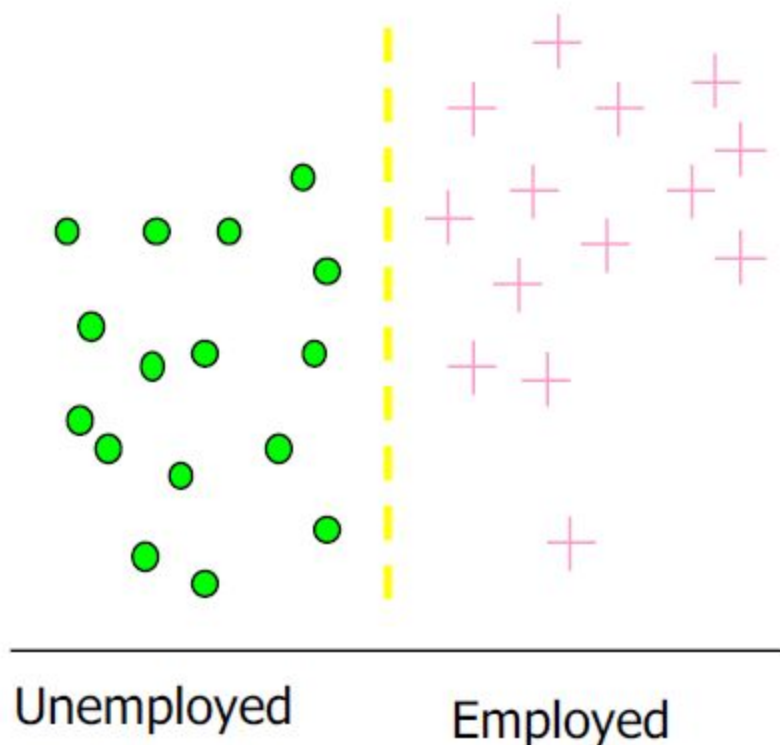
Information Gain

Which test is more informative?

**Split over whether
Balance exceeds 50K**



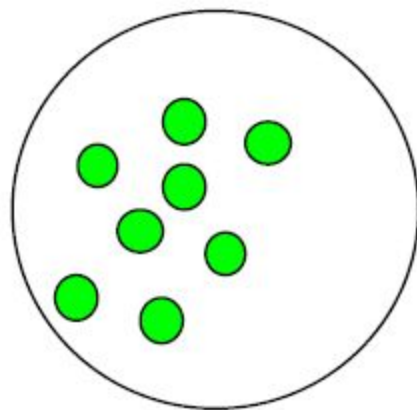
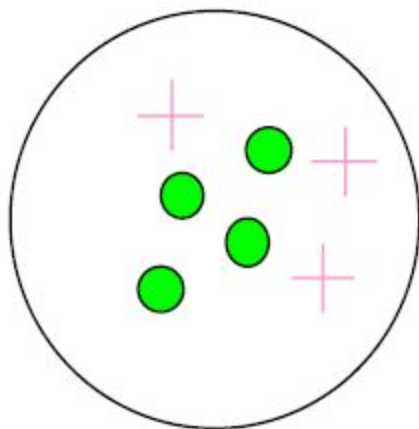
**Split over whether
applicant is employed**



Information Gain

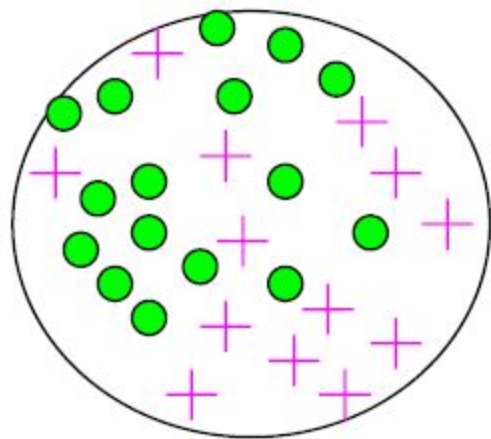
Impurity/Entropy (informal)

- Measures the level of **impurity** in a group of examples

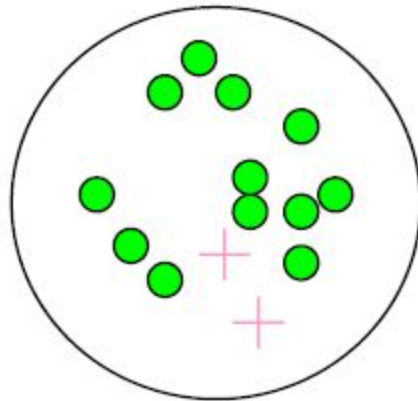


Impurity

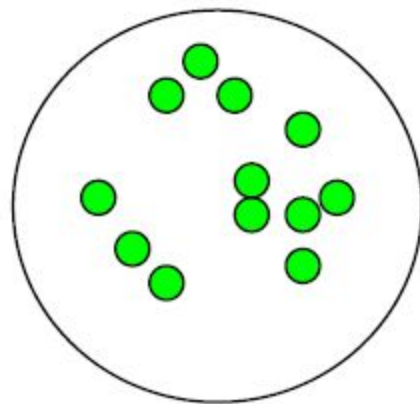
Very impure group



Less impure



**Minimum
impurity**

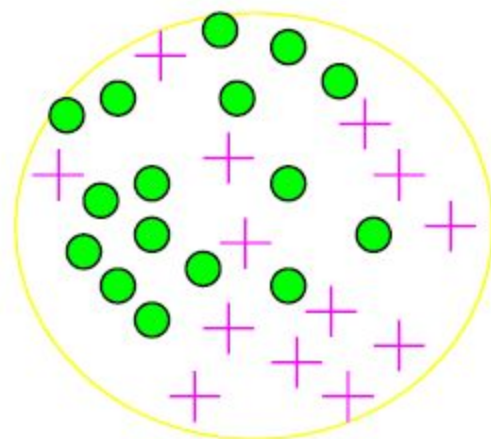


Entropy: a common way to measure impurity

- Entropy =
$$\sum_i -p_i \log_2 p_i$$

p_i is the probability of class i

Compute it as the proportion of class i in the set.



- Entropy comes from information theory. The higher the entropy the more the information content.

What does that mean for learning from examples?

2-Class Cases:

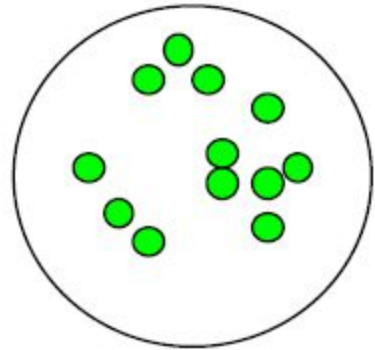
$$\text{Entropy } H(x) = - \sum_{i=1}^n P(x = i) \log_2 P(x = i)$$

- What is the entropy of a group in which all examples belong to the same class?

- entropy = $-1 \log_2 1 = 0$

not a good training set for learning

**Minimum
impurity**

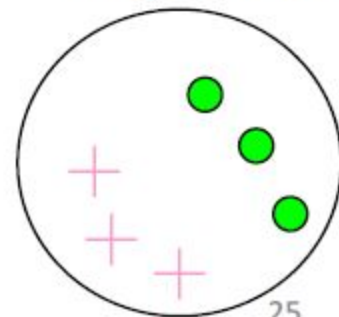


- What is the entropy of a group with 50% in either class?

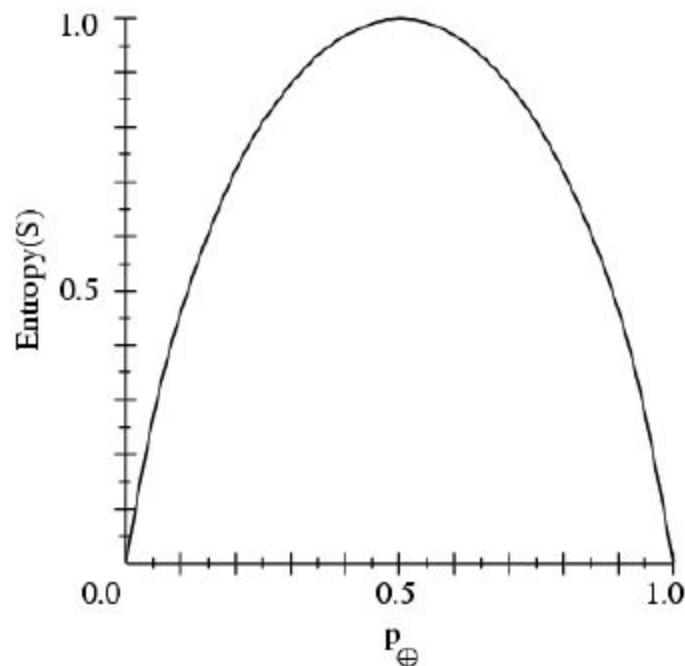
- entropy = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$

good training set for learning

**Maximum
impurity**



Sample Entropy



- S is a sample of training examples
- $p_{\oplus}$ is the proportion of positive examples in S
- $p_{\ominus}$ is the proportion of negative examples in S
- Entropy measures the impurity of S

$$H(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

Information Gain

- We want to determine **which attribute** in a given set of training feature vectors is **most useful** for discriminating between the classes to be learned.
- **Information gain** tells us how important a given attribute of the feature vectors is.
- We will use it to decide the ordering of attributes in the nodes of a decision tree.

From Entropy to Information Gain

Entropy $H(X)$ of a random variable X

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Specific conditional entropy $H(X|Y=v)$ of X given $Y=v$:

$$H(X|Y = v) = - \sum_{i=1}^n P(X = i|Y = v) \log_2 P(X = i|Y = v)$$

Conditional entropy $H(X|Y)$ of X given Y :

$$H(X|Y) = \sum_{v \in \text{values}(Y)} P(Y = v) H(X|Y = v)$$

Mutual information (aka Information Gain) of X and Y :

$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Decision Tree

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Decision Tree

- We can summarize the ID3 algorithm as illustrated below

$$\text{Entropy}(S) = \sum - p(l) \cdot \log_2 p(l)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum [p(S|A) \cdot \text{Entropy}(S|A)]$$

Entropy

We need to calculate the entropy first. Decision column consists of 14 instances and includes two labels: yes and no. There are 9 decisions labeled yes, and 5 decisions labeled no.

$$\text{Entropy}(\text{Decision}) = - p(\text{Yes}) \cdot \log_2 p(\text{Yes}) - p(\text{No}) \cdot \log_2 p(\text{No})$$

$$\text{Entropy}(\text{Decision}) = - (9/14) \cdot \log_2(9/14) - (5/14) \cdot \log_2(5/14) = 0.940$$

Now, we need to find the most dominant factor for decisioning.

Decision Tree

Wind factor on decision

$$\text{Gain}(\text{Decision}, \text{Wind}) = \text{Entropy}(\text{Decision}) - \sum [p(\text{Decision}|\text{Wind}) \cdot \text{Entropy}(\text{Decision}|\text{Wind})]$$

Wind attribute has two labels: weak and strong. We would reflect it to the formula.

$$\text{Gain}(\text{Decision}, \text{Wind}) = \text{Entropy}(\text{Decision}) - [p(\text{Decision}|\text{Wind}=\text{Weak}) \cdot \text{Entropy}(\text{Decision}|\text{Wind}=\text{Weak})] - [p(\text{Decision}|\text{Wind}=\text{Strong}) \cdot \text{Entropy}(\text{Decision}|\text{Wind}=\text{Strong})]$$

Now, we need to calculate $(\text{Decision}|\text{Wind}=\text{Weak})$ and $(\text{Decision}|\text{Wind}=\text{Strong})$ respectively.

Decision Tree

Weak wind factor on decision

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
13	Overcast	Hot	Normal	Weak	Yes

There are 8 instances for weak wind. Decision of 2 items are no and 6 items are yes as illustrated below.

$$1- \text{Entropy}(\text{Decision}|\text{Wind}=\text{Weak}) = - p(\text{No}) \cdot \log_2 p(\text{No}) - p(\text{Yes}) \cdot \log_2 p(\text{Yes})$$

$$2- \text{Entropy}(\text{Decision}|\text{Wind}=\text{Weak}) = - (2/8) \cdot \log_2 (2/8) - (6/8) \cdot \log_2 (6/8) = 0.811$$

Decision Tree

Strong wind factor on decision

Day	Outlook	Temp.	Humidity	Wind	Decision
2	Sunny	Hot	High	Strong	No
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
14	Rain	Mild	High	Strong	No

Here, there are 6 instances for strong wind. Decision is divided into two equal parts.

$$1- \text{Entropy}(\text{Decision}|\text{Wind}=\text{Strong}) = - p(\text{No}) \cdot \log_2 p(\text{No}) - p(\text{Yes}) \cdot \log_2 p(\text{Yes})$$

$$2- \text{Entropy}(\text{Decision}|\text{Wind}=\text{Strong}) = - (3/6) \cdot \log_2 (3/6) - (3/6) \cdot \log_2 (3/6) = 1$$

Now, we can turn back to Gain(Decision, Wind) equation.

$$\begin{aligned} \text{Gain}(\text{Decision}, \text{Wind}) &= \text{Entropy}(\text{Decision}) - [p(\text{Decision}|\text{Wind}=\text{Weak}) \cdot \text{Entropy}(\text{Decision}|\text{Wind}=\text{Weak})] - [p(\text{Decision}|\text{Wind}=\text{Strong}) \cdot \text{Entropy}(\text{Decision}|\text{Wind}=\text{Strong})] \\ &= 0.940 - [(8/14) \cdot 0.811] - [(6/14) \cdot 1] = 0.048 \end{aligned}$$

Decision Tree

Other factors on decision

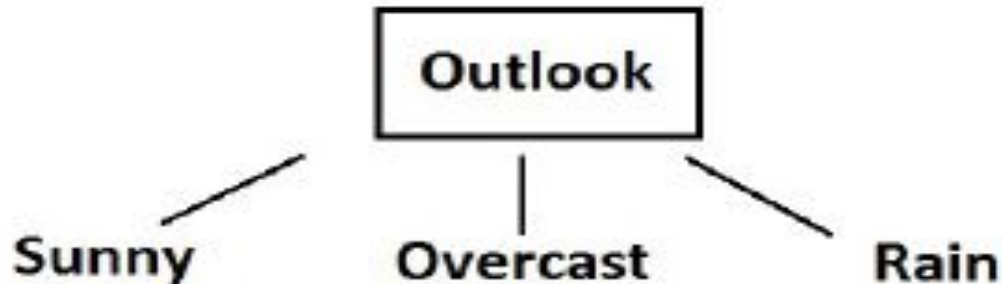
We have applied similar calculation on the other columns.

1- $\text{Gain}(\text{Decision}, \text{Outlook}) = 0.246$

2- $\text{Gain}(\text{Decision}, \text{Temperature}) = 0.029$

3- $\text{Gain}(\text{Decision}, \text{Humidity}) = 0.151$

As seen, outlook factor on decision produces the highest score. That's why, outlook decision will appear in the root node of the tree.



Decision Tree

Overcast outlook on decision

Basically, decision will always be yes if outlook were overcast.

Day	Outlook	Temp.	Humidity	Wind	Decision
3	Overcast	Hot	High	Weak	Yes
7	Overcast	Cool	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes

Decision Tree

Sunny outlook on decision

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Here, there are 5 instances for sunny outlook. Decision would be probably 3/5 percent no, 2/5 percent yes.

1- $\text{Gain}(\text{Outlook}=\text{Sunny}|\text{Temperature}) = 0.570$

2- $\text{Gain}(\text{Outlook}=\text{Sunny}|\text{Humidity}) = 0.970$

3- $\text{Gain}(\text{Outlook}=\text{Sunny}|\text{Wind}) = 0.019$

Now, humidity is the decision because it produces the highest score if outlook were sunny.

Decision Tree

At this point, decision will always be no if humidity were high.

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No

On the other hand, decision will always be yes if humidity were normal

Day	Outlook	Temp.	Humidity	Wind	Decision
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Finally, it means that we need to check the humidity and decide if outlook were sunny.

Decision Tree

Rain outlook on decision

Day	Outlook	Temp.	Humidity	Wind	Decision
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
10	Rain	Mild	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

1- Gain(Outlook=Rain | Temperature)

2- Gain(Outlook=Rain | Humidity)

3- Gain(Outlook=Rain | Wind)

Here, wind produces the highest score if outlook were rain. That's why, we need to check wind attribute in 2nd level if outlook were rain.

Decision Tree

So, it is revealed that decision will always be yes if wind were weak and outlook were rain.

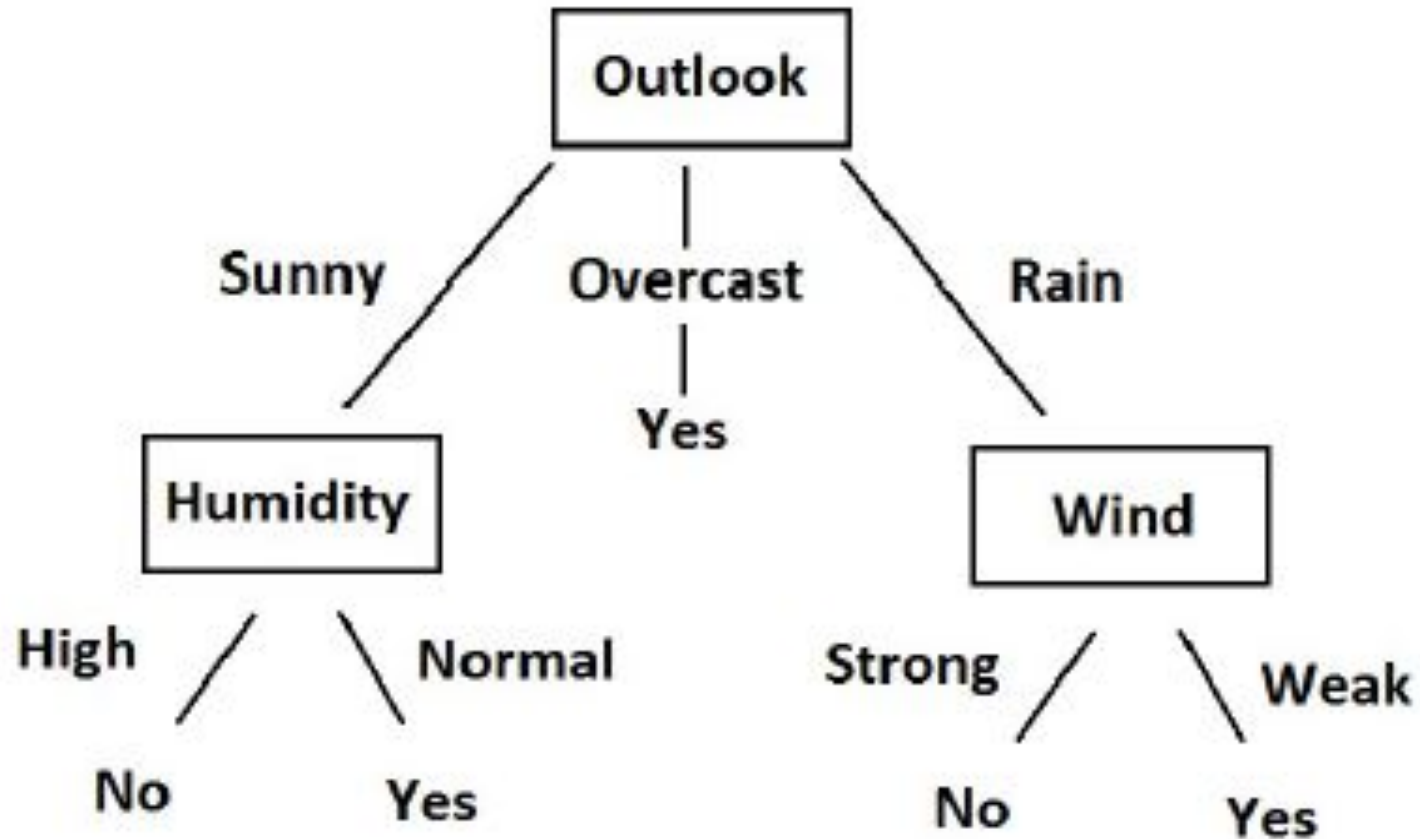
Day	Outlook	Temp.	Humidity	Wind	Decision
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes

What's more, decision will be always no if wind were strong and outlook were rain.

Day	Outlook	Temp.	Humidity	Wind	Decision
6	Rain	Cool	Normal	Strong	No
14	Rain	Mild	High	Strong	No

So, decision tree construction is over. We can use the following rules for decisioning.

Decision Tree



Final version of decision tree

Extensions of the Decision Tree Learning Algorithm

- Using gain ratios
- Real-valued data
- Noisy data and overfitting
- Generation of rules
- Setting parameters
- Cross-validation for experimental validation of performance
- C4.5 is an extension of ID3 that accounts for unavailable values, continuous attribute value ranges, pruning of decision trees, rule derivation, and so on

Using Gain Ratios

- The information gain criterion favors attributes that have a large number of values
 - If we have an attribute D that has a distinct value for each record, then $\text{Info}(D, T)$ is 0, thus $\text{Gain}(D, T)$ is maximal
- To compensate for this Quinlan suggests using the following ratio instead of Gain:

$$\text{GainRatio}(D, T) = \text{Gain}(D, T) / \text{SplitInfo}(D, T)$$

- $\text{SplitInfo}(D, T)$ is the information due to the split of T on the basis of value of categorical attribute D

$$\text{SplitInfo}(D, T) = I(|T_1|/|T|, |T_2|/|T|, \dots, |T_m|/|T|)$$

where $\{T_1, T_2, \dots, T_m\}$ is the partition of T induced by value of D

Computing Gain Ratio

■ Class P: buys_computer = "yes"

■ Class N: buys_computer = "no"

$$Info(D) = I(9,5) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940$$

age	p _i	n _i	I(p _i , n _i)
<=30	2	3	0.971
31...40	4	0	0
>40	3	2	0.971

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$\frac{5}{14} I(2,3)$ means "age <=30" has 5 out of 14 samples, with 2 yes'es and 3 no's. Hence

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

Computing Gain Ratio

- ▶ Information gain measure is biased towards attributes with a large number of values
- ▶ C4.5 (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

- ▶ $GainRatio(A) = Gain(A)/SplitInfo(A)$
- ▶ Ex. $SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) = 1.557$
 - ▶ $gain_ratio(income) = 0.029/1.557 = 0.019$
- ▶ The attribute with the maximum gain ratio is selected as the splitting attribute

Choosing the Best Attribute

- The key problem is choosing which attribute to split a given set of examples
- Some possibilities are:
 - **Random:** Select any attribute at random
 - **Least-Values:** Choose the attribute with the **smallest number of possible values**
 - **Most-Values:** Choose the attribute with **the largest number of possible values**
 - **Max-Gain:** Choose the attribute that has the **largest expected information gain**—i.e., the attribute that will result in the smallest expected size of the subtrees rooted at its children
- The ID3 algorithm uses the Max-Gain method of selecting the best attribute

Measuring Model Quality

- How good is a model?
 - Predictive accuracy
 - False positives / false negatives for a given cutoff threshold
 - Loss function (accounts for cost of different types of errors)
 - Area under the (ROC) curve
 - Minimizing loss can lead to problems with overfitting
- Training error
 - Train on all data; measure error on all data
 - Subject to overfitting (of course we'll make good predictions on the data on which we trained!)
- Regularization
 - Attempt to avoid overfitting
 - Explicitly minimize the complexity of the function while minimizing loss. Tradeoff is modeled with a *regularization parameter*

Cross-Validation

- Holdout cross-validation:
 - Divide data into training set and test set
 - Train on training set; measure error on test set
 - Better than training error, since we are measuring *generalization to new data*
 - To get a good estimate, we need a reasonably large test set
 - But this gives less data to train on, reducing our model quality!

Cross-Validation, cont.

- k-fold cross-validation:
 - Divide data into k folds
 - Train on $k-1$ folds, use the k th fold to measure error
 - Repeat k times; use average error to measure generalization accuracy
 - Statistically valid and gives good accuracy estimates
- Leave-one-out cross-validation (LOOCV)
 - k -fold cross validation where $k=N$ (test data = 1 instance!)
 - Quite accurate, but also quite expensive, since it requires building N models

Summary: Decision Tree Learning

- Inducing decision trees is one of the most widely used learning methods in practice
- Can out-perform human experts in many problems
- Strengths include
 - Fast
 - Simple to implement
 - Can convert result to a set of easily interpretable rules
 - Empirically valid in many commercial products
 - Handles noisy data
- Weaknesses include:
 - Univariate splits/partitioning using only one attribute at a time so limits types of possible trees
 - Large decision trees may be hard to understand
 - Requires fixed-length feature vectors
 - Non-incremental (i.e., batch method)